Paper Report

SIL765: Networks & System Security
Semester 2, 2025-26

1 Group Details

Group Members:

- Saharsh Laud 2024MCS2002
- Bhuvnesh Kumar 2023MCS2011

2 Paper Details

Title: Performance Implications of Packet Filtering with Linux eBPF

Authors: Dominik Scholz, Daniel Raumer, Paul Emmerich, Alexander Kurtz, Krzysztof Lesiak, Georg Carle

Publication Venue: 30th International Teletraffic Congress (ITC 2018)

Publication Date: September 2018

DOI/Link: `https://ieeexplore.ieee.org/document/8493077`

Number of Citations: 75 (as of February 2026)

Paper Type: Original Research (Performance Evaluation)

3 Project Title

DefenseNet: Behavioral eBPF Firewall with Real-Time Response

4 Problem Statement

4.1 Background

Traditional packet filtering solutions like iptables and nftables operate at the network stack level, introducing significant performance overhead due to late-stage packet interception and multiple kernel-userspace context switches. While the selected paper demonstrates that eBPF/XDP-based filtering achieves superior performance through early packet processing at the driver level, current implementations focus primarily on static rule-based filtering without intelligent threat detection capabilities.

4.2 Identified Limitations

1. **Static Filtering:** Existing eBPF firewalls rely on pre-configured static rules (IP blacklists, port filters) that cannot adapt to dynamic attack patterns.

2. **Manual Management:** Threat response requires manual rule updates, leading to delayed mitigation of active attacks.
3. **Limited Visibility:** Lack of real-time monitoring and analytics makes it difficult to understand attack patterns and system behavior.
4. **IPv4-Only Focus:** Many implementations lack comprehensive IPv6 support despite modern networks operating dual-stack.

4.3 Problem Definition

How can we extend high-performance eBPF/XDP packet filtering to incorporate **intelligent behavioral analysis** and **automated threat response** while maintaining the performance benefits demonstrated in the base paper? Specifically, can we detect DDoS attacks through rate monitoring, automatically block malicious sources, and provide comprehensive real-time visibility without significantly impacting throughput or latency?

5 Proposed Solutions/Extensions

Our project extends the paper's static eBPF filtering with four key enhancements:

5.1 Rate-Based Attack Detection

Objective: Detect volumetric DDoS attacks through behavioral traffic analysis.

Implementation:

- Maintain per-source IP statistics in eBPF hash maps tracking packet counts and timestamps
- Calculate packets-per-second rates in real-time within XDP hook
- Configure threshold values for normal vs. attack traffic (e.g., 1000 pps)
- Automatically drop packets when rate limits are exceeded
- Support configurable thresholds per rule or globally

Technical Details:

```
struct rate_info {
    __u64 packet_count;
    __u64 last_timestamp;
    __u32 pps_current;
};

// BPF_MAP_TYPE_LRU_HASH for automatic cleanup
struct bpf_map_def rate_map = {...};
```

Advantage: Provides first line of defense against flood attacks without requiring packet content inspection.

5.2 Automated Threat Mitigation

Objective: Eliminate manual intervention in threat response through dynamic blacklisting.

Implementation:

- Userspace daemon monitors eBPF map statistics continuously
- Automatically adds offending IPs to blacklist when violations detected
- Implements temporary bans with configurable timeout periods (e.g., 5 minutes)
- Maintains audit log of all automated blocking actions
- Provides manual override and whitelist capabilities

Technical Details:

- Python daemon using BCC (BPF Compiler Collection) library
- Reads rate statistics from eBPF maps every second
- Updates blacklist map atomically from userspace
- Periodic cleanup of expired entries

Advantage: Reduces response time from minutes (manual) to seconds (automated).

5.3 Enhanced Web-Based Management Interface

Objective: Provide comprehensive real-time visibility and intuitive rule management.

Dashboard Features:

- Real-time traffic statistics (total packets, blocked packets, pass rate)
- Live traffic graphs using Chart.js (packets/sec over time)
- Top blocked IPs with block counts and reasons
- Active attack detection alerts
- System performance metrics (CPU usage, map sizes)

Rule Management:

- Interactive forms for adding IP/subnet/port/protocol rules
- Rate limit configuration per rule
- Rule priority ordering
- Bulk import/export (JSON format)
- IPv4 and IPv6 unified interface

Analytics:

- Historical attack data visualization
- Block reason distribution (rate limit, blacklist, port filter)

- Traffic pattern analysis
- Exportable reports (CSV, JSON)

Technology Stack:

- Backend: Python Flask (lightweight, easy integration with BCC)
- Frontend: Bootstrap + Chart.js (responsive design)
- Real-time updates: AJAX polling (1-second refresh)

Advantage: Makes powerful eBPF filtering accessible to administrators without deep kernel knowledge.

5.4 IPv6 Protocol Support

Objective: Extend filtering capabilities to modern dual-stack networks.

Implementation:

- Parse both IPv4 and IPv6 headers in XDP program
- Maintain separate or unified maps for IPv6 addresses
- Support IPv6 CIDR notation for subnet filtering
- Rate limiting for IPv6 sources
- Unified web interface managing both protocols

Technical Details:

```
// Detect protocol
if (eth->h_proto == bpf_htons(ETH_P_IP)) {
    // IPv4 processing
} else if (eth->h_proto == bpf_htons(ETH_P_IPV6)) {
    // IPv6 processing
}
```

Advantage: Comprehensive protection for modern networks, future-proof implementation.

5.5 Exploratory: Machine Learning Integration

Objective: Investigate ML-based anomaly detection for adaptive security.

Potential Approach:

- Extract traffic features from eBPF maps (packet sizes, inter-arrival times, protocol distribution)
- Train lightweight classification models (decision trees, random forests)
- Classify traffic as benign vs. attack patterns
- Use ML predictions to dynamically adjust rate thresholds
- Detect zero-day attacks not covered by static rules

Status: Exploratory component – will assess feasibility during implementation phase.

6 Evaluation Metrics

6.1 Security Effectiveness

- **Detection Accuracy:** Percentage of actual attacks correctly identified
- **False Positive Rate:** Percentage of legitimate traffic incorrectly blocked
- **Response Time:** Time from attack detection to mitigation (target: <5 seconds)
- **Attack Coverage:** Types of attacks successfully mitigated (SYN flood, UDP flood, port scans)

6.2 Performance Impact

- **Throughput:** Packets/second processed with extensions vs. baseline
  - Target: <10% degradation compared to paper's 10 Mpps baseline
- **Latency:** Median and 99th percentile packet processing latency
  - Target: <100 μs median (comparable to paper's results)
- **CPU Utilization:** CPU usage under various traffic loads
- **Memory Overhead:** eBPF map memory consumption

6.3 Scalability

- **Rule Capacity:** Maximum number of rules before performance degradation
- **Connection Tracking:** Number of simultaneous flows tracked
- **Map Size Impact:** Performance with varying blacklist sizes

6.4 Usability

- **Configuration Time:** Time to add/modify rules via web interface
- **System Responsiveness:** Web dashboard latency and update frequency

6.5 Testing Methodology

- **Traffic Generation:** iperf3, tcpreplay, hping3 for attack simulation
- **Network Setup:** Direct 10 GbE connection between generator and device under test (DUT)
- **Attack Scenarios:**
  - SYN flood (varying intensities: 1K, 10K, 100K pps)
  - UDP flood
  - Port scanning
  - Mixed legitimate + attack traffic
- **Baseline Comparison:** Static iptables, static eBPF (from previous work), our adaptive eBPF

7 Key Takeaways

7.1 Expected Contributions

1. **Practical Security Enhancement:** Transform eBPF from static filter to intelligent defense system
2. **Automated Response:** Demonstrate feasibility of self-defending networks without manual intervention
3. **Performance Validation:** Prove that behavioral analysis can be added without sacrificing eBPF's performance benefits
4. **Operational Visibility:** Show that kernel-level security can be made accessible through intuitive interfaces

7.2 Technical Learnings

- Hands-on experience with eBPF/XDP programming and kernel-level packet processing
- Understanding trade-offs between security features and performance overhead
- Practical knowledge of DDoS mitigation techniques and rate limiting algorithms
- Integration of kernel-space eBPF with userspace management tools

7.3 Real-World Impact

- Deployable solution for protecting high-speed networks (10+ Gbps)
- Reduced operational overhead through automation
- Faster threat response compared to traditional manual firewall management
- Foundation for future ML-enhanced network security systems

7.4 Potential Challenges

- eBPF verifier restrictions (no unbounded loops, instruction limits)
- Race conditions in map updates between kernel and userspace
- Optimal threshold configuration to minimize false positives
- Memory management for large-scale connection tracking

7.5 Success Criteria

The project will be considered successful if we achieve:

All four core extensions implemented and functional

Detection accuracy >95% for tested attack types

False positive rate <1%

Performance within 10% of baseline static filtering

Working web interface with real-time monitoring

References

1. Scholz, D., Raumer, D., Emmerich, P., Kurtz, A., Lesiak, K., & Carle, G. (2018). Performance Implications of Packet Filtering with Linux eBPF. *30th International Teletraffic Congress (ITC 30)*, 209–217.
2. Linux Kernel Documentation: BPF and XDP. `https://www.kernel.org/doc/html/latest/bpf/`
3. BPF Compiler Collection (BCC). `https://github.com/iovisor/bcc`
4. Høiland-Jørgensen, T., et al. (2018). The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. *ACM CoNEXT*.
5. Cloudflare Blog: L4Drop - XDP DDoS Mitigations. `https://blog.cloudflare.com/l4drop-xdp-ebpf-based-ddos-mitigations/`